Playwright Automation

8-Day Training Program: Beginner to Medium-Advanced (TypeScript)

Practice sites (free)

- https://www.saucedemo.com (UI e-commerce app)
- https://demo.playwright.dev/todomvc (Todo app)
- https://the-internet.herokuapp.com (alerts, iframes, uploads, windows)
- https://restful-booker.herokuapp.com (REST API)

Daily routine

About 1 hour theory, 2 hours hands-on practice, 30 minutes revision.

Schedule at a glance

- **Day 1:** Setup and first test
- **Day 2:** Locators, actions and assertions
- **Day 3:** Test structure, hooks and configuration
- **Day 4:** Page Object Model, fixtures, data-driven tests
- **Day 5:** API testing and network control
- **Day 6:** Real-world UI scenarios
- **Day 7:** Debugging, reporting and CI/CD
- **Day 8:** Advanced topics and capstone project

Day 1: Setup and Your First Test

Topics: Architecture (browser, context, page), install, project structure, running tests, headed vs headless, codegen.

```
npm init playwright@latest
npx playwright test                 # run all
npx playwright test --headed        # see the browser
npx playwright codegen https://www.saucedemo.com   # record actions
npx playwright show-report
```

Practice script: tests/day1-login.spec.ts

```
import { test, expect } from '@playwright/test';

test('valid user can login', async ({ page }) => {
  await page.goto('https://www.saucedemo.com');
  await page.fill('#user-name', 'standard_user');
  await page.fill('#password', 'secret_sauce');
  await page.click('#login-button');
  await expect(page).toHaveURL(/inventory/);
  await expect(page.locator('.title')).toHaveText('Products');
});
```

Task: Write a negative test with the locked_out_user and assert the error message.

Day 2: Locators, Actions and Assertions

Topics: getByRole, getByText, getByLabel, getByPlaceholder, getByTestId, CSS/XPath, chaining and filtering, auto-waiting, web-first assertions, dropdowns, checkboxes, keyboard and mouse actions.

```
import { test, expect } from '@playwright/test';

test('todo actions and assertions', async ({ page }) => {
  await page.goto('https://demo.playwright.dev/todomvc');
  const input = page.getByPlaceholder('What needs to be done?');

  for (const item of ['Learn Playwright', 'Write tests', 'Run in CI']) {
    await input.fill(item);
    await input.press('Enter');
  }

  const items = page.getByTestId('todo-item');
  await expect(items).toHaveCount(3);

  await items.filter({ hasText: 'Write tests' }).getByRole('checkbox').check();
  await expect(items.nth(1)).toHaveClass(/completed/);

  await page.getByRole('link', { name: 'Completed' }).click();
  await expect(items).toHaveCount(1);
});
```

Task: On SauceDemo, sort products by 'Price (low to high)' using selectOption and assert the first item is the cheapest.

Key rule: Avoid waitForTimeout. Use web-first assertions (expect(locator)...) because they retry automatically.

Day 3: Test Structure, Hooks and Configuration

Topics: describe, beforeEach/afterEach/beforeAll, test.step, tags, skip/fixme/only, playwright.config.ts (baseURL, timeouts, retries, workers, projects), parallel execution, environment variables.

playwright.config.ts

```
import { defineConfig, devices } from '@playwright/test';

export default defineConfig({
  testDir: './tests',
  timeout: 30_000,
  retries: process.env.CI ? 2 : 0,
  workers: process.env.CI ? 2 : undefined,
  reporter: [['html'], ['list']],
  use: {
    baseURL: 'https://www.saucedemo.com',
    trace: 'on-first-retry',
    screenshot: 'only-on-failure',
    video: 'retain-on-failure',
  },
  projects: [
    { name: 'chromium', use: { ...devices['Desktop Chrome'] } },
    { name: 'firefox',  use: { ...devices['Desktop Firefox'] } },
  ],
});
```

Tests with hooks and steps

```
import { test, expect } from '@playwright/test';

test.describe('Cart @smoke', () => {
  test.beforeEach(async ({ page }) => {
    await page.goto('/');
    await page.fill('#user-name', 'standard_user');
    await page.fill('#password', 'secret_sauce');
    await page.click('#login-button');
  });

  test('add item to cart', async ({ page }) => {
    await test.step('Add backpack', async () => {
      await page.click('[data-test="add-to-cart-sauce-labs-backpack"]');
    });
    await test.step('Verify badge', async () => {
      await expect(page.locator('.shopping_cart_badge')).toHaveText('1');
    });
  });
});

npx playwright test --grep @smoke --project=chromium
```

Task: Add a 'remove from cart' test and run the suite in both browsers with 3 workers.

Day 4: Page Object Model, Fixtures and Data-Driven Testing

Topics: POM design, custom fixtures, parameterized tests, reading test data from JSON.

pages/LoginPage.ts

```
import { Page, Locator } from '@playwright/test';

export class LoginPage {
  readonly username: Locator;
  readonly password: Locator;
  readonly loginBtn: Locator;
  readonly error: Locator;

  constructor(private page: Page) {
    this.username = page.locator('#user-name');
    this.password = page.locator('#password');
    this.loginBtn = page.locator('#login-button');
    this.error = page.locator('[data-test="error"]');
  }
  async goto() { await this.page.goto('/'); }
  async login(u: string, p: string) {
    await this.username.fill(u);
    await this.password.fill(p);
    await this.loginBtn.click();
  }
}
```

fixtures/base.ts

```
import { test as base } from '@playwright/test';
import { LoginPage } from '../pages/LoginPage';

export const test = base.extend<{ loginPage: LoginPage }>({
  loginPage: async ({ page }, use) => {
    const lp = new LoginPage(page);
    await lp.goto();
    await use(lp);
  },
});
export { expect } from '@playwright/test';
```

tests/login-data.spec.ts

```
import { test, expect } from '../fixtures/base';

const cases = [
  { user: 'locked_out_user', pass: 'secret_sauce', msg: 'locked out' },
  { user: 'standard_user',   pass: 'wrong',        msg: 'do not match' },
  { user: '',                pass: '',             msg: 'Username is required' },
];

for (const c of cases) {
  test(`invalid login: "${c.user}" / "${c.pass}"`, async ({ loginPage }) => {
    await loginPage.login(c.user, c.pass);
    await expect(loginPage.error).toContainText(c.msg);
  });
}
```

Task: Create InventoryPage and CartPage classes and write an end-to-end purchase flow using fixtures only (no raw selectors in the test file).

Day 5: API Testing and Network Control

Topics: request fixture, CRUD API tests, API + UI hybrid tests, page.route for mocking, intercepting and waiting for responses.

```
import { test, expect } from '@playwright/test';

const API = 'https://restful-booker.herokuapp.com';

test('API: create and fetch booking', async ({ request }) => {
  const create = await request.post(`${API}/booking`, {
    data: {
      firstname: 'Ram', lastname: 'QA', totalprice: 500, depositpaid: true,
      bookingdates: { checkin: '2026-10-01', checkout: '2026-10-05' },
    },
  });
  expect(create.ok()).toBeTruthy();
  const { bookingid } = await create.json();

  const get = await request.get(`${API}/booking/${bookingid}`);
  expect(get.status()).toBe(200);
  expect((await get.json()).firstname).toBe('Ram');
});

test('UI: mock an API response', async ({ page }) => {
  await page.route('**/api/products', route =>
    route.fulfill({ json: [{ id: 1, name: 'Mocked Product' }] })
  );
  await page.goto('/products');           // your app under test
  await expect(page.getByText('Mocked Product')).toBeVisible();
});

test('wait for a specific response', async ({ page }) => {
  const [resp] = await Promise.all([
    page.waitForResponse(r => r.url().includes('/search') && r.status() === 200),
    page.getByRole('button', { name: 'Search' }).click(),
  ]);
  expect(resp.ok()).toBeTruthy();
});
```

Task: Write API tests for PUT/DELETE, and use page.route to simulate a 500 error, then assert the UI error message.

Day 6: Real-World UI Scenarios

Topics: Login once and reuse storageState, iframes, alerts and dialogs, file upload and download, multiple tabs and popups, hover, drag and drop, shadow DOM.

auth.setup.ts

```
import { test as setup } from '@playwright/test';

setup('authenticate', async ({ page }) => {
  await page.goto('https://www.saucedemo.com');
  await page.fill('#user-name', 'standard_user');
  await page.fill('#password', 'secret_sauce');
  await page.click('#login-button');
  await page.context().storageState({ path: '.auth/user.json' });
});

// playwright.config.ts projects:
// { name: 'setup', testMatch: /auth\.setup\.ts/ },
// { name: 'chromium', use: { storageState: '.auth/user.json' },
//   dependencies: ['setup'] },
```

Dialogs, upload, new tab, iframe

```
import { test, expect } from '@playwright/test';

test('handle dialog', async ({ page }) => {
  page.once('dialog', d => d.accept());
  await page.goto('https://the-internet.herokuapp.com/javascript_alerts');
  await page.getByText('Click for JS Confirm').click();
  await expect(page.locator('#result')).toHaveText('You clicked: Ok');
});

test('file upload', async ({ page }) => {
  await page.goto('https://the-internet.herokuapp.com/upload');
  await page.setInputFiles('#file-upload', 'testdata/sample.txt');
  await page.click('#file-submit');
  await expect(page.locator('#uploaded-files')).toHaveText('sample.txt');
});

test('new tab', async ({ page, context }) => {
  await page.goto('https://the-internet.herokuapp.com/windows');
  const [newPage] = await Promise.all([
    context.waitForEvent('page'),
    page.getByText('Click Here').click(),
  ]);
  await expect(newPage.locator('h3')).toHaveText('New Window');
});

test('iframe', async ({ page }) => {
  await page.goto('https://the-internet.herokuapp.com/iframe');
  const editor = page.frameLocator('#mce_0_ifr').locator('#tinymce');
  await editor.fill('Hello from Playwright');
});
```

Task: Automate a file download using page.waitForEvent('download') and verify the file name.

Day 7: Debugging, Reporting and CI/CD

Topics: Trace Viewer, UI mode, --debug, Inspector, screenshots and videos, HTML/Allure reports, retries and flaky-test handling, GitHub Actions, Docker.

```
npx playwright test --ui                    # interactive UI mode
npx playwright test --debug                 # step through with the inspector
npx playwright show-trace test-results/<folder>/trace.zip
npx playwright test --last-failed

test('custom screenshot and attachment', async ({ page }, testInfo) => {
  await page.goto('/');
  const shot = await page.screenshot({ fullPage: true });
  await testInfo.attach('home-page', { body: shot, contentType: 'image/png' });
});
```

.github/workflows/playwright.yml

```
name: Playwright Tests
on: [push, pull_request]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: 20 }
      - run: npm ci
      - run: npx playwright install --with-deps
      - run: npx playwright test
      - uses: actions/upload-artifact@v4
        if: always()
        with:
          name: playwright-report
          path: playwright-report/
```

Task: Deliberately break a test, open its trace, and find the failing step. Push your project to GitHub and get the workflow to run green.

Day 8: Medium-Advanced Topics and Capstone

Topics: Visual regression testing, accessibility checks, mobile emulation, geolocation and permissions, test sharding, custom reporters, best practices for a scalable framework.

```
// Visual comparison
test('homepage visual', async ({ page }) => {
  await page.goto('https://playwright.dev');
  await expect(page).toHaveScreenshot('home.png', { maxDiffPixelRatio: 0.02 });
});
// npx playwright test --update-snapshots   (create or refresh baselines)

// Mobile emulation
import { devices } from '@playwright/test';
test.use({ ...devices['iPhone 13'] });

test('mobile view', async ({ page }) => {
  await page.goto('https://playwright.dev');
  await expect(page.getByRole('button', { name: /navigation/i })).toBeVisible();
});

// Accessibility (npm i -D @axe-core/playwright)
import AxeBuilder from '@axe-core/playwright';

test('a11y scan', async ({ page }) => {
  await page.goto('https://playwright.dev');
  const results = await new AxeBuilder({ page }).analyze();
  expect(results.violations).toEqual([]);
});

// Sharding across CI machines:  npx playwright test --shard=1/3
```

Capstone project (SauceDemo)

- POM classes for Login, Inventory, Cart and Checkout
- Custom fixture with authenticated state
- Data-driven checkout tests from a JSON file
- One API + UI hybrid test
- Tags (@smoke, @regression) and multi-browser config
- HTML report and a GitHub Actions pipeline

Best Practices Checklist

- Prefer getByRole / getByTestId over brittle CSS/XPath.
- Never use hard waits. Rely on auto-wait and web-first assertions.
- Keep tests independent (no test depends on another's data).
- Keep logic in page objects and fixtures, and assertions in tests.
- Use storageState and API setup to avoid repeating UI login.
- Trace on retry, and treat flaky tests as bugs.